



Hugging Face Transformers

সম্পূর্ণ বাংলা নোট — NLP Practical Guide

pipeline · Tokenizer · Fine-tuning · Trainer API



📋 বিষয়সূচি

1. সহজ ভাষায় পরিচিতি (Intuition)
2. বিস্তারিত ব্যাখ্যা (Deep Explanation)
3. Math / Theory
4. Code এর ব্যাখ্যা (Python)
5. Visual / Diagram
6. Real-world Use Cases
7. Pros & Cons
8. Common Mistakes & Gotchas
9. Related Topics
10. Memory Tricks



সহজ ভাষায় পরিচিতি (Intuition)

► Hugging Face কী?

ধরো তুমি রান্না করতে চাও, কিন্তু মশলা বাটতে জানো না। কেউ একজন আগে থেকে মশলা বেটে রেখে দিয়েছে — তুমি শুধু নিয়ে রান্না করো। **Hugging Face** ঠিক এই কাজ করে — লক্ষ লক্ষ data দিয়ে আগে থেকে train করা model রেখে দিয়েছে, তুমি শুধু নামিয়ে ব্যবহার করো।

► Hugging Face ইকোসিস্টেম

অংশ	কী করে
Models Hub	হাজার হাজার pre-trained model
Datasets	ready-made dataset
Spaces	demo deploy করা যায়
Transformers Library	এই library দিয়েই model use করি

► কেন দরকার ?

- Scratch থেকে model train করা অনেক সময় ও GPU লাগে
- Hugging Face থেকে ready model নামিয়ে কাজ শুরু করা যায়
- Fine-tuning করে নিজের কাজে fit করা যায়



বিস্তারিত ব্যাখ্যা (Deep Explanation)

► pipeline() — সবচেয়ে সহজ API

`pipeline` হলো একটা **function** (class না)। এটা call করলে ভেতরে একটা object তৈরি হয়:

```
pipeline("text-classification")
↓
Hugging Face Hub থেকে default model download
↓
TextClassificationPipeline object তৈরি হয়
↓
সেই object এ আছে: Tokenizer + Model + Postprocessor
```

💡 **classifier(...)** কীভাবে কাজ করে?

`classifier("text")` মানে আসলে `classifier.__call__("text")` — Python এ object কে function এর মতো call করা যায় যদি `__call__` method থাকে। এটাকে বলে **callable object**।

► **Tokenizer — text → number** রূপান্তর

Model কখনো সরাসরি text বোঝে না। Tokenizer text কে numbers এ convert করে:

```
"I love food" → Tokenizer → [101, 1045, 2293, 2833, 102] → Model
```

Vocabulary: AutoTokenizer এর ভেতরে একটি বিশাল dictionary আছে। এই dictionary তৈরি হয়েছে Wikipedia/Books থেকে **WordPiece** algorithm দিয়ে:

```
Common word → একটাই token: 'cooking' → 8343
Rare word → ভেঙে যায়: 'unhappiness' → ['un', '##happy', '##ness']
(## মানে word এর মাঝের অংশ)
```

► **Tokenizer Output** এর তিনটা অংশ

Field	মানে
input_ids	প্রতিটা token এর ID number
attention_mask	1=আসল token, 0=padding (ignore)
token_type_ids	0=Sentence A, 1=Sentence B

► **Padding** কেন দরকার?

Model একসাথে batch (অনেক sentence) process করে। সব sentence সমান লম্বা না হলে `[PAD]` token যোগ করে সমান করে। `attention_mask` দিয়ে বলে কোনগুলো ignore করতে হবে:

```
Sentence 1: "I love food [PAD] [PAD] [PAD]"
attention: [ 1, 1, 1, 0, 0, 0 ]
```



Model এখানে মনোযোগ দেবে না

► **Logits → Probability → Label**

Model থেকে raw output আসে যার নাম **logits** — এটা সরাসরি বোঝা যায় না। Softmax দিয়ে probability তে convert করতে হয়:

```
logits:  [-3.4,  3.6]  ← raw score
      ↓ softmax
probs:  [0.0009, 0.9991] ← probability (মোট = 1)
      ↓ argmax (সবচেয়ে বড়টার index)
index:           1
      ↓ id2label
label:  "POSITIVE" ✓
```

► **token_type_ids — দুটো Sentence**

BERT কে দুটো sentence একসাথে দেওয়া যায়, মূলত QA তে লাগে। একটি sentence দিলে সব 0, দুটো দিলে A=0, B=1:

```
[CLS] What is cooking? [SEP] Cooking is an art. [SEP]
0 0 0 0 0 1 1 1 1
```

► **Fine-tuning কী?**

```
Pre-trained Model (লক্ষ লক্ষ text পড়া)
      ↓ নিজের data দিয়ে আরো train
Custom Model (নিজের কাজে perfect) ✓
```

পুরো scratch থেকে train না করে, already শেখা model কে নতুন কাজ শেখানো।



Math / Theory

► Softmax Formula

$$P(i) = e^{(z_i)} / \sum(e^{(z_j)} \text{ for all } j)$$

যেখানে:

z_i = logit value (model এর raw output)

e = Euler's number (~2.718)

সব output এর sum = 1 (এটাই probability)

Manual Calculation:

```
logits = [-3.4, 3.6]
```

```
e^(-3.4) = 0.033
```

```
e^(3.6) = 36.6
```

```
sum = 36.633
```

```
P(NEGATIVE) = 0.033 / 36.633 = 0.0009 (0.09%)
```

```
P(POSITIVE) = 36.6 / 36.633 = 0.9991 (99.91%) ✓
```

Python

► Cross Entropy Loss (Training এ ব্যবহার)

$$L = - \sum(y_i \times \log(y_{\hat{i}}))$$

যেখানে:

y_i = সঠিক উত্তর (0 বা 1)

$y_{\hat{i}}$ = model এর predicted probability

Loss বড় → model বেশি ভুল

Loss ছোট → model ভালো শিখেছে

► Weight Update (Gradient Descent)

```
new_weight = old_weight - (learning_rate × gradient)
```

```
lr = 2e-5 = 0.00002
```

lr বড় → fast কিন্তু unstable ⚠️

lr ছোট → slow কিন্তু stable ✅

8



Code এর ব্যাখ্যা (Python)

► pipeline() দিয়ে Text Classification

```
from transformers import pipeline # pipeline function import

# pipeline call করলে TextClassificationPipeline object তৈরি হয়
classifier = pipeline("text-classification")

result = classifier("I love cooking delicious food!") # __call__() চলে
print(result)
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

Python

► pipeline() দিয়ে Summarization

```
summarizer = pipeline("summarization") # summarization model load

text = """
Hugging Face is a company that provides tools for building machine learning models.
Their transformers library has become the standard for NLP tasks across the industry.
Thousands of pre-trained models are available on their Hub platform for free.
"""
```

Python

```
result = summarizer(text, max_length=50, min_length=20) # summary বানাও
print(result[0]['summary_text'])
```

► **pipeline()** দিয়ে Question Answering

```
qa = pipeline("question-answering") # QA model load

context = """
The Smart Cooking System uses an ESP32 microcontroller connected to a FastAPI backend.
The LLM layer uses Mistral-7B for intelligent recipe reasoning and ingredient management.
"""

result = qa(
    question="What LLM is used in the Smart Cooking System?",
    context=context # এখান থেকে answer খুঁজবে
)
print(result['answer'])
# 'Mistral-7B'
```

► **Tokenizer manually** ব্যবহার

```
from transformers import AutoTokenizer # AutoTokenizer import

# bert-base-uncased model এর tokenizer download করো
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "I love cooking delicious food!"

tokens = tokenizer(text) # tokenize করো
print(tokens)
# {'input_ids': [101, 1045, 2293, 8343, 12090, 2833, 999, 102],
#  'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1]}

# প্রতিটা token আলাদা দেখতে
print(tokenizer.tokenize(text))
# ['i', 'love', 'cooking', 'delicious', 'food', '!']

# numbers থেকে আবার text
ids = [101, 1045, 2293, 2833, 102]
print(tokenizer.decode(ids))
# [CLS] i love food [SEP]
```

► Padding সহ Batch Tokenize

```
Python
sentences = [
    "I love food",                    # ছোট sentence
    "I love cooking very delicious food"  # বড় sentence
]

tokens = tokenizer(
    sentences,
    padding=True,                    # ছোট sentence এ [PAD] যোগ করো
    return_tensors="pt"              # PyTorch tensor হিসেবে দাও
)

print(tokens['input_ids'])
# tensor([
#   [101, 1045, 2293, 2833, 102,    0,    0,    0], ← শেষে 0 = padding
#   [101, 1045, 2293, 8343, 2200, 12090, 2833, 102] ← কোনো padding নেই
# ])

print(tokens['attention_mask'])
# tensor([
#   [1, 1, 1, 1, 1, 0, 0, 0], ← 0 মানে model ignore করবে
#   [1, 1, 1, 1, 1, 1, 1, 1] ← সব 1 মানে সব real token
# ])
```

► দুটো Sentence একসাথে (QA style)

```
Python
sentence_a = "What is cooking?"      # প্রশ্ন (Sentence A)
sentence_b = "Cooking is an art."    # context (Sentence B)

result = tokenizer(sentence_a, sentence_b) # pair হিসেবে দাও
print(result['token_type_ids'])
# [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
# ← Sentence A      ← Sentence B

# Batch এ pair দিতে চাইলে দুটো list দাও
sentences_a = ["What is cooking?", "Who made BERT?"]
sentences_b = ["Cooking is an art.", "Google made BERT."]
result = tokenizer(sentences_a, sentences_b, padding=True)
```

► Model manually load ও Predict

```
Python
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
```



```

# model ও tokenizer download করো
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")

text = "I love cooking delicious food!"
inputs = tokenizer(text, return_tensors="pt") # pt = PyTorch tensor

with torch.no_grad(): # gradient calculate করতে হবে না (predict mode)
    outputs = model(**inputs) # model এ input দাও

print(outputs.logits)
# tensor([[ -3.4,  3.6]]) ← raw score

import torch.nn.functional as F

probs = F.softmax(outputs.logits, dim=1) # logits → probability
print(probs)
# tensor([[0.0009, 0.9991]])

predicted = torch.argmax(probs) # সবচেয়ে বড়টার index
labels = model.config.id2label # index → label mapping
print(labels[predicted.item()])
# POSITIVE

```

► Fine-tuning — Dataset Class

```

import torch
from torch.utils.data import Dataset # PyTorch এর base Dataset class

class TextDataset(Dataset): # Dataset inherit করে নিজের class বানাই
    def __init__(self, texts, labels, tokenizer): # একবারই চলে শুরুতে
        self.encodings = tokenizer(
            texts,
            padding=True, # সব sentence সমান লম্বা করো
            truncation=True, # বেশি লম্বা হলে কেটে দাও
            return_tensors="pt" # PyTorch tensor হিসেবে দাও
        )
        self.labels = labels # [1, 0, 1, 0, 1, 0] সংরক্ষণ করো

    def __len__(self): # DataLoader জিজ্ঞেস করে "কয়টা item?"
        return len(self.labels)

    def __getitem__(self, idx): # idx নম্বর item দাও
        return {
            'input_ids': self.encodings['input_ids'][idx], # token numbers
            'attention_mask': self.encodings['attention_mask'][idx], # real/padding
            'labels': torch.tensor(self.labels[idx]) # সঠিক উত্তর
        }

```

► Fine-tuning — Training Loop

```
from torch.utils.data import DataLoader # data কে batch বানায়
from torch.optim import AdamW          # weight update করে

dataset = TextDataset(texts, labels, tokenizer) # dataset object বানাও
loader = DataLoader(dataset, batch_size=2, shuffle=True)
# batch_size=2 → একসাথে ২টা sentence দেবে
# shuffle=True → প্রতি epoch এ data এলোমেলো করবে (মুখস্থ ঠেকাতে)

optimizer = AdamW(model.parameters(), lr=2e-5) # সব weight দাও, lr=0.00002

model.train() # training mode চালু করো (Dropout etc. active)
for epoch in range(3): # পুরো data ৩ বার দেখবে
    total_loss = 0

    for batch in loader: # প্রতিবার ২টা করে sentence আসে
        optimizer.zero_grad() # আগের batch এর gradient মুছে দাও

        outputs = model(
            input_ids=batch['input_ids'],
            attention_mask=batch['attention_mask'],
            labels=batch['labels'] # সঠিক উত্তর দিলে model নিজেই loss বের করে
        )

        loss = outputs.loss # model কতটা ভুল করল
        loss.backward() # কোন weight কতটুকু বদলাবে বের করো (backprop)
        optimizer.step() # weight আপডেট করো ← এটাই আসল learning

    total_loss += loss.item() # batch এর loss যোগ করো (.item() = tensor→number)

    print(f"Epoch {epoch+1} | Loss: {total_loss:.4f}")
    # Epoch 1 | Loss: 1.2341 ← বেশি ভুল
    # Epoch 2 | Loss: 0.8123 ↓ কমছে
    # Epoch 3 | Loss: 0.3421 ✓ অনেক কম ভুল
```

► Trainer API — সহজ উপায়

```
from transformers import Trainer, TrainingArguments # Hugging Face এর ready training system

args = TrainingArguments(
    output_dir="my-model", # trained model কোথায় save হবে
    num_train_epochs=3, # পুরো data ৩ বার দেখবে
    per_device_train_batch_size=2, # একসাথে ২টা sentence process করবে
    learning_rate=2e-5, # weight update এর হার
)

trainer = Trainer(
```

```

model=model,          # কোন model train করবে
args=args,            # উপরের settings
train_dataset=dataset, # কোন data দিয়ে train করবে
)

trainer.train() # উপরের পুরো loop এর কাজ এক লাইনে ✓

```

► Model Save ও Load

```

model.save_pretrained("my-sentiment-model") # model save করো
tokenizer.save_pretrained("my-sentiment-model") # tokenizer save করো

# পরে load করে use করো
classifier = pipeline(
    "text-classification",
    model="my-sentiment-model" # নিজের trained model
)
result = classifier("This food is absolutely amazing!")

```

Python



Visual / Diagram

► pipeline() এর ভেতরে

```

তুমি লেখো: pipeline("text-classification")
↓
Hugging Face Hub থেকে model download
↓
TextClassificationPipeline object তৈরি
↓
| Tokenizer → Model → Output |

```

► Tokenizer Flow

```
"I love food"
  ↓ tokenize
['i', 'love', 'food']
  ↓ vocabulary lookup
[1045, 2293, 2833]
  ↓ special tokens যোগ
[101, 1045, 2293, 2833, 102]
  ↑           ↑
[CLS]           [SEP]
(sentence শুরু) (sentence শেষ)
```

► Padding এর ছবি

padding=True দিলে:

```
| 101 1045 2293 2833 102 0 0 | ← ছোট sentence
| 1 1 1 1 1 0 0 | ← attention_mask (0=ignore)
| 101 1045 2293 8343 2200 2833 102 | ← বড় sentence
| 1 1 1 1 1 1 1 | ← সব 1 (সব real)
```

► Fine-tuning Flow

```
তোমার Data (texts + labels)
  ↓
TextDataset Class
  ↓
DataLoader (batch=2)
  ↓
| for each batch: |
| zero_grad() | ← আগের gradient মুছে
| model → loss | ← কতটা ভুল?
```

	loss.backward()		← কোথায় ভুল ?
	optimizer.step()		← ঠিক করো

↓

Epoch শেষে Loss কমে

↓

Model Save 

► Logits → Label (Full Flow)

Text: "I love food!"

↓ tokenizer

input_ids: [101, 1045, 2293, 2833, 999, 102]

↓ model

logits: [-3.4, 3.6]

↓ softmax

probs: [0.0009, 0.9991]

↓ argmax

index: 1

↓ id2label

"POSITIVE"  (99.91% confident)



Real-world Use Cases

1. **Customer Support Bot** — customer এর message POSITIVE/NEGATIVE classify করে priority ঠিক করা
2. **News Summarization** — বড় article ছোট করে দেখানো (BBC, Google News)

3. **Smart Search (QA)** — document থেকে সরাসরি answer বের করা (Google, Bing)
4. **Spam Detection** — email বা message spam কিনা বের করা (Gmail)
5. **E-commerce Review Analysis** — product review থেকে sentiment বের করে rating improve করা



Pros & Cons

সুবিধা ✓

অসুবিধা ✗

Pre-trained model তাই কম data লাগে

বড় model download করতে সময় লাগে

pipeline() দিয়ে ৩ লাইনে কাজ হয়

GPU ছাড়া বড় model slow

হাজার হাজার model Hub এ আছে

Fine-tuning এ RAM বেশি লাগে

PyTorch ও TensorFlow দুটোই support

Custom architecture কঠিন

Community অনেক বড়

কিছু model commercial use এ restriction



Common Mistakes & Gotchas

✗ ভুল ১: **padding=True** না দেওয়া

```
tokenizer(sentences) # ভুল — batch এ কাজ হবে না  
tokenizer(sentences, padding=True) # সঠিক ✓
```

Python

❌ ভুল ২: **return_tensors** ভুলে যাওয়া

```
inputs = tokenizer(text) # ভুল — model এ দেওয়া যাবে না
inputs = tokenizer(text, return_tensors="pt") # সঠিক ✓
```

Python

❌ ভুল ৩: **zero_grad()** ভুলে যাওয়া

```
for batch in loader:
    outputs = model(**batch) # ভুল — gradient জমতে থাকবে

for batch in loader:
    optimizer.zero_grad() # সঠিক ✓ — আগে মুছে দাও
    outputs = model(**batch)
```

Python

⚠️ ভুল ৪: **model.train()** না দেওয়া

Training এর আগে `model.train()` না দিলে Dropout বন্ধ থাকে, overfitting হয়।

💡 ভুল ৫: **Single sentence** এ **padding** আশা করা

Single sentence এ padding আসে না, batch (একাধিক sentence) দিলেই আসে।



Related Topics

► আগে জানা দরকার

Python OOP

PyTorch Tensor

Neural Network basics

Backpropagation

► পরে শেখা উচিত

LoRA / QLoRA

Hugging Face Datasets

Hugging Face Hub (upload)

PEFT Library

LLM Fine-tuning (Mistral, LLaMA)



Memory Tricks

pipeline()

রান্নার ready মশলা — নিজে কিছু করতে হয় না, শুধু use করো

Tokenizer

বাংলা থেকে মোর্স কোড — text কে number এ convert করে

attention_mask

হাজিরা খাতা — 1=আছে (দেখো), 0=নেই (ignore করো)

padding

সব বাক্স সমান সাইজ করতে ফাঁকা জায়গা ভরা

logits → softmax → argmax

raw score → percentage → winner

Fine-tuning

অভিজ্ঞ রাঁধুনিকে নতুন রেসিপি শেখানো

 ১ লাইনে সারসংক্ষেপ:

Hugging Face এ আগে থেকে train করা model আছে, `pipeline()` দিয়ে সহজে use করো,
Tokenizer text কে number এ বদলায়, আর Fine-tuning দিয়ে নিজের data তে fit করাও।

 Hugging Face Transformers — বাংলা নোট | ML/DL Study Series